TensorFlow in TPU

Architecture, Neural Networks, and Hardware Acceleration

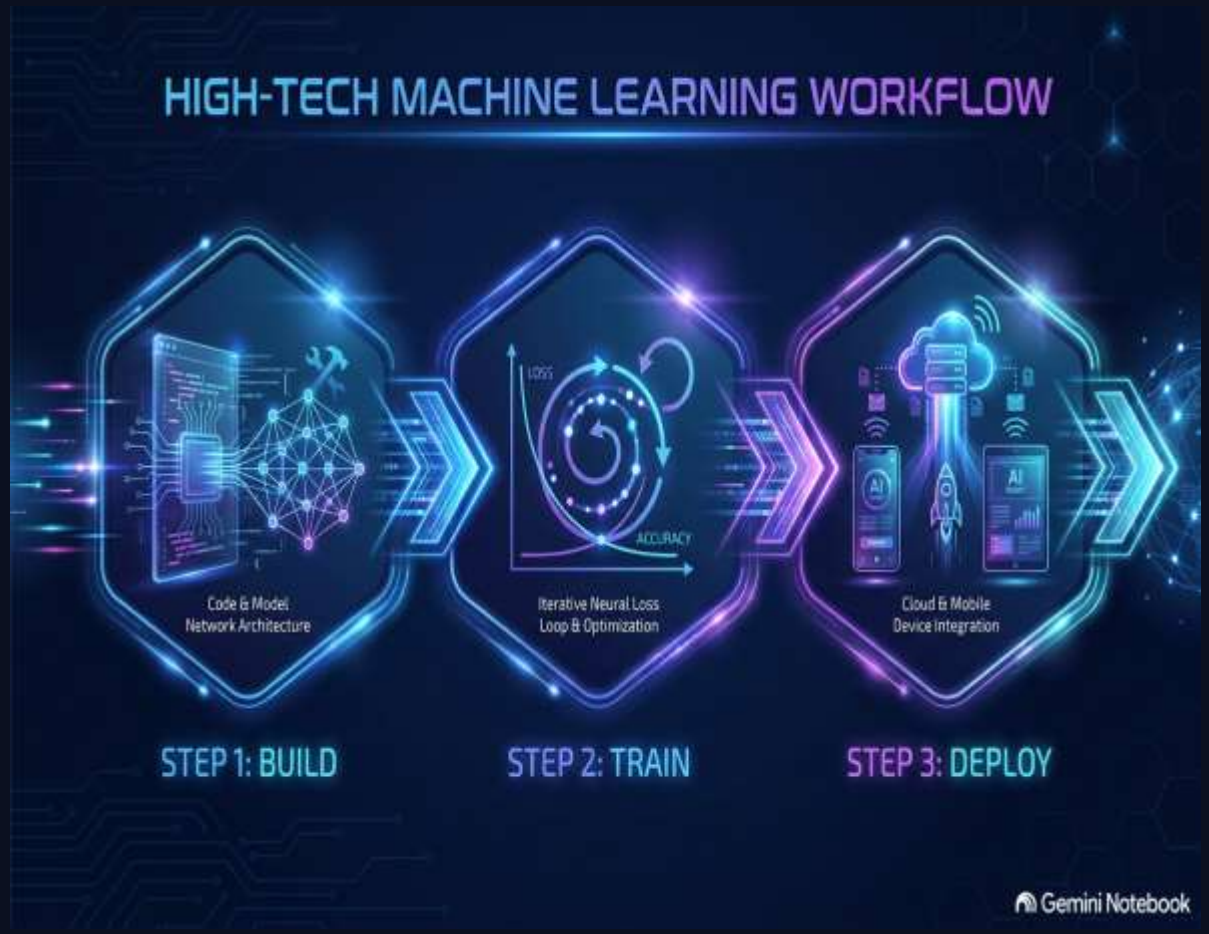


CREDITS

Dr. D. Y. Patil Institute of Technology

Department of Digital Electronics & Logic Design

End-to-End Machine Learning Lifecycle: Build, Train, and Deploy



1. BUILD

Define model topologies using high-level Keras APIs or low-level tensor ops.

2. TRAIN

Iteratively optimize weights using automatic differentiation and backpropagation.

3. DEPLOY

Serve models across Cloud Servers, Mobile (TF Lite), Edge Devices, or Web (TF.js).

Tensors Power Multidimensional Data & Computation Graphs

Scalar (0D)

Single numerical value
e.g., magnitude = 42.0

Vector (1D)

1D array of values
e.g., feature vector [1.2, 3.4]

Matrix (2D)

2D grid of values
e.g., grayscale image [28x28]

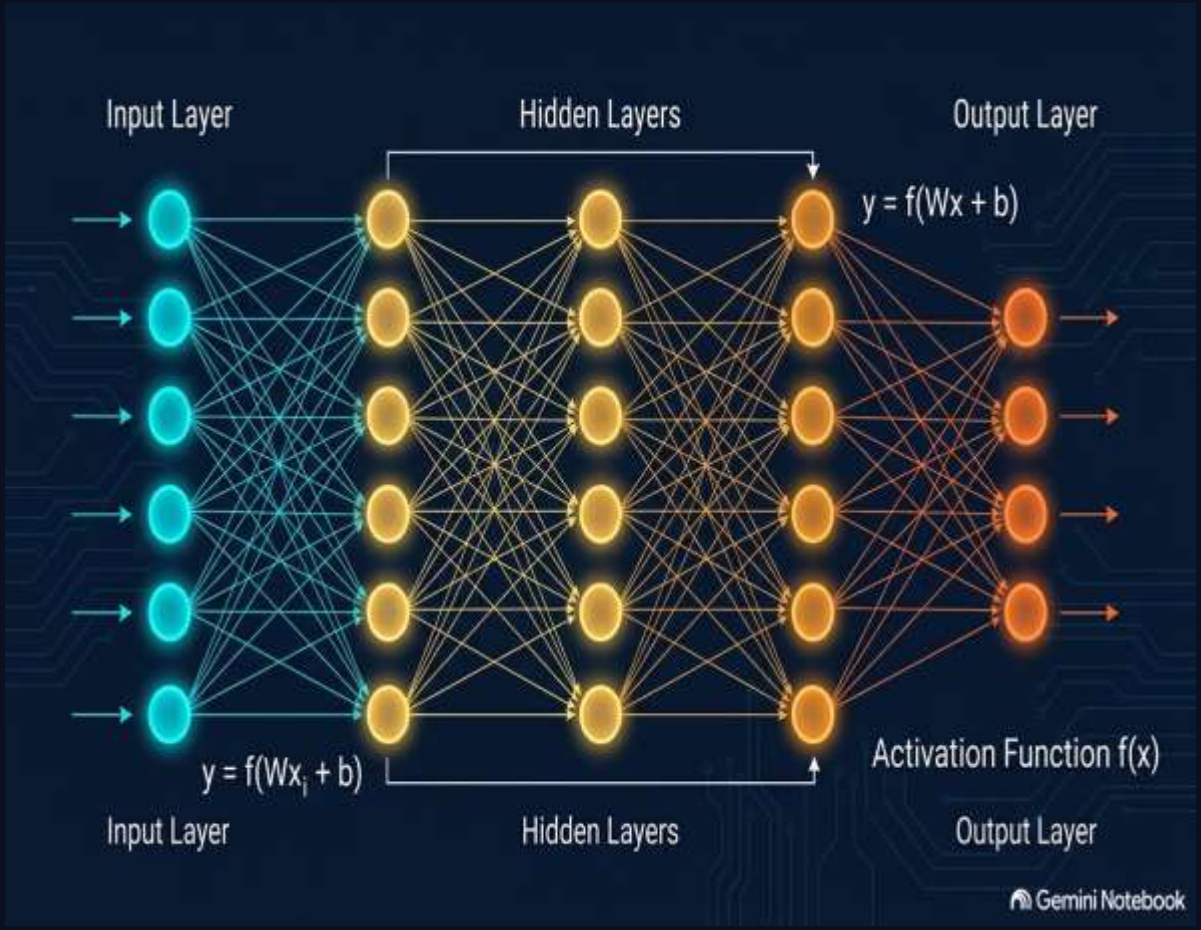
Tensor (3D+)

Multi-dimensional array
e.g., RGB image batch [32, 224, 224, 3]

EXECUTION PARADIGMS: EAGER VS. GRAPH MODE

- Eager Execution (Default): Operations evaluate immediately, matching Python imperative programming for seamless debugging.
- Graph Mode (@tf.function): Traces python code into optimized static compute graphs for maximum parallel hardware efficiency.

Neuron Computation: Weighted Sums, Activations & Backprop



NEURON MATHEMATICAL MODEL

$$y = f(W \cdot x + b)$$

4-STEP TRAINING CYCLE

1. Forward Pass: Compute predictions from inputs.
2. Loss Calculation: Measure error against ground truth.
3. Backpropagation: Compute gradients using tf.GradientTape.
4. Weight Update: Adjust W & b via Adam/SGD optimizers.

Modular Software Suite Spanning Development to Production

Keras API

High-level API for rapidly assembling layers, models, and custom loss functions.

tf.data Pipeline

High-throughput data loading, batching, shuffling, and augmentation pipelines.

TensorFlow Lite

On-device quantised model execution for iOS, Android, microcontrollers & IoT.

TensorFlow.js

In-browser training and client-side inference using WebGL hardware acceleration.

TensorBoard

Interactive visualization suite for metric tracking, graph analysis, and profiling.

TensorFlow Core

Low-level C++ engine providing automatic differentiation and hardware kernels.

Deploying Machine Learning Across Industry Domains

Healthcare & Diagnostics

Analyzing X-rays, MRI scans, and pathology images for early disease detection.

Computer Vision & Mobility

Real-time object detection, lane tracking, and pedestrian identification in autonomous driving.

Natural Language Processing

Powering translation, sentiment analysis, speech-to-text, and conversational AI models.

Smart Agriculture

Crop health evaluation, disease identification, and automated yield estimation.

Recommendation Systems

Personalized content ranking and user preference predictions for e-commerce.

Predictive Maintenance

Monitoring vibration and temperature sensor signals to prevent industrial hardware failure.

5-Stage CNN Pipeline and Executable Keras Model

CNN TRAINING WORKFLOW

1. Dataset Ingestion (tf.keras.utils.image_dataset_from_directory)
2. Normalization & Augmentation (Rescale, Flip, Zoom)
3. Model Architecture (Conv2D -> MaxPooling -> Dense)
4. Model Compilation (Adam Optimizer, CrossEntropy Loss)
5. Model Evaluation & Quantized Export

```
# Executable Keras CNN Architecture

import tensorflow as tf
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Input(shape=(128, 128, 3)),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(2, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Bridging High-Level Software Graphs to Physical Digital Datapaths

1. SOFTWARE GRAPH

Abstract computation graph representing tensor operations, activations, and layer topologies.

2. XLA COMPLIER

Accelerated Linear Algebra (XLA) fuses ops, optimizes memory layout, and generates hardware machine code.

3. LOGIC DATAPATH

Physical execution across multipliers, adders, registers, and memory buses using MAC operations.

```
CORE MAC OPERATION:
ACC <= ACC + (A × B)
```

Tensor Processing Units: Matrix Acceleration via Systolic Arrays



- **Systolic Array Matrix Unit (MXU)**
  Houses 256x256 Processing Elements (PEs) executing 65,536 Multiply-Accumulate (MAC) operations per cycle.

- **Zero-Overhead Register Pipelining**
  Data flows directly through neighboring PEs without repeatedly reading and writing to main memory registers.

- **High-Bandwidth Memory (HBM)**
  Interconnected memory buses deliver multi-terabyte bandwidth required for large deep learning models.

- **Domain-Specific Hardware Logic**
  Custom ASIC designed specifically to eliminate general-purpose CPU/GPU control overhead.

Summary: Unifying Software Models with Intelligent Hardware Logic

ABSTRACTION

TensorFlow decouples complex neural mathematical models from physical hardware implementation.

ARITHMETIC

Transformations boil down to parallel tensor operations and Multiply-Accumulate (MAC) math.

ACCELERATION

Dedicated ASICs like TPUs achieve orders-of-magnitude speedups via specialized systolic datapaths.

Presented for Digital Electronics & Logic Design
Dr. D. Y. Patil Institute of Technology